AVR-MAP BACKEND ARCHITECTURE & ENGINEERING SPECIFICATION

SIH26053 — Adaptive Variable-Resolution 2.5D LiDAR Mapping

System: AVR-Map
Frontend: AVR Console
Document: Backend Architecture & Engineering Specification
Version: 1.0
Target: SIH 2026 Prototype
Runtime: Ubuntu 24.04 + ROS 2 Jazzy
Primary Language: C++17/20
AI Runtime: Python + PyTorch initially
Status: Implementation Source of Truth

---

1. Purpose

This document defines the complete backend architecture of AVR-Map.

It specifies:

- process architecture;
- ROS 2 architecture;
- software package boundaries;
- LiDAR data ingestion;
- timestamps and coordinate systems;
- semantic inference;
- adaptive map engine;
- multi-resolution data structure;
- cell aggregation;
- semantic fusion;
- overhang representation;
- small-object preservation;
- dynamic-object tracking;
- traversability;
- frontend communication;
- APIs;
- binary protocols;
- configuration;
- diagnostics;
- storage;
- experiments;
- benchmarking;
- concurrency;
- memory management;
- fault tolerance;

- testing;
- deployment;
- security;
- performance optimisation.

The backend SHALL be treated as the primary technical product.

The frontend visualizes its state.

---

2. Backend Mission

The backend must continuously transform:

```
RAW LIDAR
   ↓
GEOMETRIC OBSERVATIONS
   ↓
SEMANTIC UNDERSTANDING
   ↓
MOTION UNDERSTANDING
   ↓
ADAPTIVE RESOLUTION DECISIONS
   ↓
HIERARCHICAL 2.5D MAP
   ↓
TRAVERSABILITY + SAFETY INFORMATION
```

while satisfying:

```
LOW LATENCY

BOUNDED MEMORY

REAL-TIME OPERATION

DETERMINISTIC SPATIAL OWNERSHIP

FAILURE ISOLATION

REPRODUCIBILITY

OBSERVABILITY
```

---

3. Most Important Backend Principle

The system SHALL have two worlds:

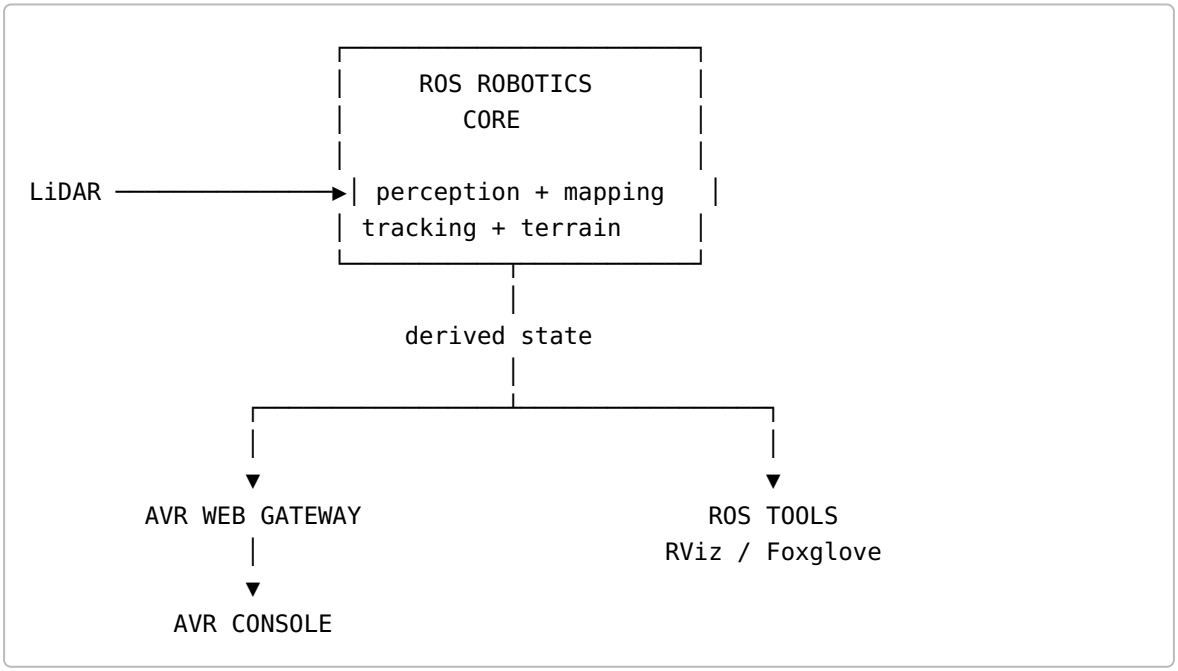
```
REAL-TIME ROBOTICS CORE

and

APPLICATION / VISUALIZATION WORLD
```

They must never become tightly coupled.

Architecture:



The frontend must NEVER become part of the perception loop.

---

4. Explicit Architecture Rule

These components must be able to crash without crashing the map:

```
AVR Console

Web gateway

Foxglove

RViz
```

```
benchmark database

run recorder
```

Conversely:

```
LiDAR processing

pose transforms

mapping

terrain state
```

must continue whenever technically possible.

---

5. System Layers

AVR backend consists of five architectural layers.

```
┌──────────────────────────────────────────┐
│  L5 — APPLICATION / EXPERIMENTATION      │
│  Gateway, benchmark runner, storage      │
├──────────────────────────────────────────┤
│  L4 — ROS INTEGRATION                    │
│  Nodes, topics, services, lifecycle      │
├──────────────────────────────────────────┤
│  L3 — PERCEPTION                         │
│  Semantics, tracking, uncertainty        │
├──────────────────────────────────────────┤
│  L2 — SPATIAL INTELLIGENCE               │
│  Adaptive map, terrain, traversability   │
├──────────────────────────────────────────┤
│  L1 — DOMAIN / MATH                      │
│  Morton, cells, geometry, algorithms     │
└──────────────────────────────────────────┘
```

Lower layers SHALL NOT depend on higher ones.

---

6. Critical Architecture Decision

Core algorithms should be implemented as:

Pure C++ libraries independent of ROS wherever practical.

ROS nodes become adapters around the libraries.

Bad architecture:

```
ROS callback
   ↓
2000-line mapping algorithm
```

Correct architecture:

```
ROS MESSAGE
     ↓
ROS ADAPTER
     ↓
SemanticFrame
     ↓
PURE C++ MAP ENGINE
     ↓
MapUpdateResult
     ↓
ROS ADAPTER
```

Benefits:

- easier unit testing;
- deterministic testing without ROS;
- easier profiling;
- easier future integration;
- less middleware coupling;
- potential standalone use.

---

7. Complete Process Architecture

Recommended V1 deployment:

```
PROCESS 1
avr_sensor_container
    ├── input component
    └── preprocessing component


PROCESS 2
avr_semantic_node
```

```
    └── Python / GPU inference


PROCESS 3
avr_core_container
    ├── adaptive mapper
    ├── tracker
    ├── traversability
    └── metrics


PROCESS 4
avr_gateway
    └── HTTP + WebSocket + UI mirror


PROCESS 5
avr_supervisor


OPTIONAL PROCESS 6
foxglove_bridge


OPTIONAL PROCESS 7
rosbag2 recorder
```

---

8. Why Not Put Everything in One Process?

Because neural inference can fail because of:

```
CUDA

PyTorch

GPU OOM

model dependency mismatch
```

and should not take down mapping.

Likewise the browser gateway should not affect robotics execution.

---

9. Why Compose C++ Nodes?

ROS 2 supports composable C++ nodes in the same process.

Composition can avoid message marshalling/unmarshalling overhead, and ROS documentation demonstrates lower-latency intra-process pipelines.

Therefore:

```
mapping
tracking
traversability
metrics
```

should preferably reside inside:

```
avr_core_container
```

using intra-process communication where beneficial.

---

10. Inter-Process Communication

For large messages between processes:

```
PointCloud2
SemanticPointCloud
```

use ROS 2 DDS.

On the same workstation, shared-memory transport should be enabled/verified.

Fast DDS supports shared-memory transport specifically to reduce copying and OS/network overhead for same-host communication.

---

11. Middleware Policy

Primary:

```
Fast DDS
```

with shared-memory operation verified.

Fallback:

```
Cyclone DDS
```

if hardware-specific testing shows better stability/performance.

Middleware selection SHALL remain configurable.

Never build map logic that depends on one DDS implementation.

---

12. Source Repository Structure

Recommended complete project:

```
AVR-Map/
│
├── backend/
│   └── ros2_ws/
│       └── src/
│
├── frontend/
│
├── models/
│
├── config/
│
├── datasets/
│
├── tools/
│
├── docker/
│
├── results/
│
├── docs/
│
└── scripts/
```

---

13. Backend Package Structure

```
backend/ros2_ws/src/
│
├── avr_msgs/
```

```
│
├── avr_common/
│
├── avr_spatial/
│
├── avr_mapping_core/
│
├── avr_tracking_core/
│
├── avr_traversability_core/
│
├── avr_input/
│
├── avr_preprocess/
│
├── avr_semantics/
│
├── avr_mapping_ros/
│
├── avr_tracking_ros/
│
├── avr_traversability_ros/
│
├── avr_metrics/
│
├── avr_gateway/
│
├── avr_supervisor/
│
└── avr_bringup/
```

---

14. `avr_common`

Contains only shared fundamentals:

```
FrameContext

timestamps

semantic enum

status enum

math utilities

configuration structures
```

```
logging utilities

profiling utilities
```

Must not know anything about HTTP/frontend.

---

15. `avr_spatial`

Pure C++.

Responsible for:

```
CellKey

TileKey

Morton coding

floor division

coordinate quantisation

parent/child relationships

hierarchical neighbourhood queries
```

This is one of the most thoroughly unit-tested packages.

---

16. `avr_mapping_core`

Pure C++.

Contains:

```
AdaptiveMap

TileStore

ResolutionController

FrameAggregator

SemanticFusion
```

```
ElevationFusion

VerticalProfile

ReplayCache

MapMaintenance

MapDeltaBuilder
```

This is the heart of AVR-Map.

---

17. `avr_tracking_core`

Pure C++.

Contains:

```
DBSCAN / clustering

detection extraction

Kalman Filter

assignment cost

Hungarian association

track lifecycle
```

---

18. `avr_traversability_core`

Pure C++.

Contains:

```
neighbourhood extraction

terrain-plane fitting

slope

roughness
```

```
step height

clearance analysis

semantic cost

uncertainty cost

vehicle feasibility

traversability score
```

---

19. `avr_msgs`

Contains all custom ROS messages and services.

It must contain no algorithms.

---

20. Canonical Semantic Ontology

Different datasets use different class IDs.

The map engine must never depend directly on:

```
SemanticKITTI label IDs

RELLIS IDs

GOOSE IDs
```

Instead introduce a canonical ontology.

Initial example:

```
enum class SemanticClass : uint16_t
{
    UNKNOWN = 0,

    ROAD,
    TERRAIN,
    VEGETATION,

    BUILDING,
    POLE,
```

```
    BARRIER,

    VEHICLE,
    PERSON,
    CYCLIST,

    OTHER_OBSTACLE
};
```

---

21. Dataset Label Mapping

Each dataset has:

```
semantic_kitti_labels.yaml

rellis3d_labels.yaml

goose_labels.yaml
```

Example:

```
source: SemanticKITTI

mapping:
  10: VEHICLE
  11: CYCLIST
  30: PERSON
  40: ROAD
  50: BUILDING
  70: VEGETATION
```

Actual mappings SHALL be verified against each dataset.

---

22. Semantic Categories

Backend should additionally classify canonical semantics into:

```
TERRAIN

STATIC_STRUCTURE

DYNAMIC_CAPABLE_OBJECT
```

```
SMALL_HAZARD

UNKNOWN
```

This prevents duplicating class logic throughout algorithms.

---

23. Frame Context

Every processed LiDAR frame SHALL have:

```
struct FrameContext
{
    uint64_t frame_seq;

    int64_t sensor_timestamp_ns;

    Eigen::Isometry3d T_odom_lidar;

    bool pose_valid;

    bool semantics_valid;

    bool deskewed;
};
```

---

24. Three Different Clocks

The backend must distinguish:

Sensor / ROS Time

Used for:

```
data alignment

TF lookup

dataset playback
```

Monotonic Time

Use:

```
std::chrono::steady_clock
```

for latency measurement.

Wall Clock

Used only for:

```
human-readable logs

run timestamps
```

Never measure pipeline latency with system wall clock.

---

25. Input Pipeline

```
LiDAR / dataset
      ↓
timestamp verification
      ↓
point validation
      ↓
ROI crop
      ↓
deskew optional
      ↓
TF transform
      ↓
filtered point cloud
      ↓
semantic provider
```

---

26. Invalid Point Policy

Reject:

```
NaN

Inf

outside configured XYZ bounds
```

```
malformed point records
```

Count rejected points in metrics.

Do not crash because of malformed sensor data.

---

27. Pose Synchronisation

For every LiDAR scan:

```
lookup transform at sensor timestamp
```

using tf2.

If transform unavailable within:

```
tf_timeout_ms
```

the frame SHALL NOT be inserted into the persistent map.

It MAY still be published for raw visualization.

---

28. Why This Matters

Inserting a point cloud at the wrong vehicle pose can smear:

```
walls

curbs

vehicles

terrain boundaries
```

across the map.

A missing frame is better than a spatially corrupted map.

---

29. Deskew

If the sensor supplies:

```
per-point time
```

and odometry is available during scan acquisition:

```
deskew_enabled = true
```

SHOULD compensate vehicle motion through the scan.

Otherwise document:

```
deskew = disabled
```

for that experiment.

---

30. Semantic Provider Architecture

The semantic node exposes one conceptual interface:

```
class SemanticProvider:
    def infer(self, frame):
        ...
```

Implementations:

```
GroundTruthProvider

SalsaNextProvider

FutureSparseCNNProvider
```

---

31. Ground Truth Mode

Ground-truth semantics must remain a first-class runtime mode.

This allows us to test:

```
adaptive mapping itself
```

without asking whether a neural-network mistake caused the result.

---

32. AI Mode

Initial neural path:

```
filtered LiDAR
      ↓
range projection
      ↓
SalsaNext
      ↓
pixel semantics
      ↓
back-project labels to original points
      ↓
SemanticPointCloud
```

SalsaNext was designed as an uncertainty-aware LiDAR semantic segmentation system, making it useful for the prototype's uncertainty-driven refinement experiments.

---

33. Inference Optimisation

The semantic node SHALL use:

```
model.eval()

torch.inference_mode()
```

and SHOULD evaluate:

```
FP16 inference
```

when validated to preserve sufficient prediction quality.

Batch size:

```
1
```

for real-time latency.

Do not batch several sensor frames simply to improve GPU throughput.

---

34. Model Warm-Up

At activation:

```
load model

allocate GPU buffers

run several warm-up passes
```

before the node reports:

```
READY
```

Benchmark timings must exclude model initialization.

---

35. AI Backpressure

Semantic inference queue SHALL be bounded.

Recommended:

```
capacity = 2
```

If the model becomes slower than incoming LiDAR:

```
DROP STALE UNPROCESSED FRAME
```

rather than accumulate seconds of delay.

For autonomous perception:

> latest information is usually more valuable than complete stale information.

---

36. Semantic Output Format

Use:

```
sensor_msgs/PointCloud2
```

with fields:

```
x
y
z
intensity

semantic_label
confidence
uncertainty
```

Point ordering SHOULD match the input filtered cloud.

---

37. Core Internal Representation

The map should not repeatedly parse `PointCloud2`.

On receiving semantic data:

```
PointCloud2
     ↓
ONE PARSE
     ↓
SemanticFrame SoA
```

Example:

```
struct SemanticFrame
{
    FrameContext context;

    std::vector<float> x;
    std::vector<float> y;
    std::vector<float> z;

    std::vector<uint16_t> semantic;
```

```
    std::vector<uint8_t> confidence;

    std::vector<uint8_t> uncertainty;
};
```

---

38. Why Structure-of-Arrays for Frames?

Algorithms frequently need:

```
all x

all z

all semantic labels
```

rather than complete object structures one at a time.

SoA improves:

```
cache behaviour

SIMD opportunities

GPU transfer opportunities
```

and works naturally with the frontend binary protocol later.

---

39. Buffer Reuse

Do not allocate 100k-element vectors every frame.

Use:

```
reserve()

reusable frame buffers

double buffering
```

Typical strategy:

```
buffer A → processing

buffer B → receiving next frame

swap
```

---

40. Map Coordinate System

The hierarchical grid is permanently anchored to:

```
odom origin
```

It does NOT move its mathematical grid origin with the vehicle.

Instead:

```
active window follows vehicle

grid coordinates remain stable
```

This is important.

It prevents map cells from changing identity merely because the vehicle moved.

---

41. Base Resolution

```
r0 = 0.05 m
```

Hierarchy:

```
L0 = 0.05 m
L1 = 0.10 m
L2 = 0.20 m
L3 = 0.40 m
```

General:

```
r(L) = r0 × 2^L
```

---

42. Base Cell Coordinates

Given world/odom position:

```
x, y
```

define:

```
bx = floor((x - origin_x) / r0)

by = floor((y - origin_y) / r0)
```

These are integer coordinates at the finest possible resolution.

---

43. Level Coordinates

For hierarchy level `L`:

```
scale = 2^L

cx = floor_div(bx, scale)

cy = floor_div(by, scale)
```

`floor_div` MUST behave correctly for negative coordinates.

---

44. Boundary Convention

Cells always use:

```
[min, max)
```

half-open intervals.

Exactly one cell owns every coordinate.

---

45. Spatial Key

Logical:

```
struct CellKey
{
    uint64_t morton_xy;
    uint8_t level;
};
```

Coordinates are ZigZag encoded before Morton interleaving.

---

46. Parent Relationship

If cell:

```
(level=L, cx, cy)
```

then parent:

```
level = L + 1

parent_x = floor_div(cx, 2)

parent_y = floor_div(cy, 2)
```

---

47. Children

For a cell at level `L`:

```
children level = L - 1
```

coordinates:

```
(2cx,     2cy)

(2cx+1,   2cy)

(2cx,     2cy+1)

(2cx+1,   2cy+1)
```

---

48. Tile Architecture

Cells are allocated in tiles.

Default:

```
16 × 16 cells
```

for one hierarchy level.

```
Tile
├── 256 cell slots
├── active mask
├── dirty mask
├── child mask
├── last touched frame
└── level metadata
```

---

49. Why Tile-Based Storage?

Hashing every map cell independently creates significant overhead.

Tiles provide:

```
fewer hash lookups

contiguous memory

better cache locality

cheap iteration

cheap dirty tracking

easy pruning

future GPU compatibility
```

---

50. Tile Store

Initial implementation:

```
std::unordered_map<TileKey, Tile*>
```

combined with:

```
TilePool
```

for memory reuse.

Do not introduce exotic hash-table libraries until profiling demonstrates a meaningful need.

---

51. Tile Pool

When rolling-map tiles expire:

Bad:

```
delete

later malloc new tile
```

Repeatedly.

Better:

```
retire tile
     ↓
reset tile
     ↓
return to pool
     ↓
reuse later
```

This reduces heap churn.

---

52. Cell Ownership Invariant

A location may belong to:

```
one active parent
```

OR:

```
active children
```

Never both.

Research specifically identifies single-leaf ownership as necessary to prevent double counting across hierarchical levels.

---

53. Map Update Is Transactional Per Frame

A map frame update SHALL behave conceptually as:

```
PREPARE

→

COMMIT

→

PUBLISH REVISION
```

Consumers must never see half-updated hierarchy state.

---

54. Map Revision

Every successful committed map update increments:

```
uint64 map_revision
```

Example:

```
812
813
814
815
```

Revision IDs are used by:

```
gateway

frontend
```

```
snapshot logic

debugging

benchmarking
```

---

55. Frame Processing Pipeline

For semantic frame `F`:

```
1. validate frame

2. canonicalize semantics

3. classify terrain / structure / object points

4. detect dynamic-capable clusters

5. update tracks

6. calculate requested resolution

7. batch hierarchy changes

8. apply required splits

9. replay previous points where split occurred

10. bin current points into final active leaves

11. update ground/elevation statistics

12. update semantics/hazards

13. update vertical profiles

14. update object/dynamic layer

15. calculate/update traversability

16. evaluate merge candidates

17. perform bounded maintenance

18. prune old/outside tiles
```

```
19. create map delta

20. commit map revision

21. publish outputs

22. insert frame into replay cache
```

---

56. Resolution Controller Inputs

For each region:

```
distance

semantic importance

semantic uncertainty

local point density

dynamic state

height variance

terrain complexity
```

V2 can add:

```
vehicle trajectory

collision risk

vehicle speed
```

---

57. Baseline Distance Level

Default:

```
0–10 m
→ L0 / 5cm

10–30 m
→ L1 / 10cm
```

```
30–60 m
→ L2 / 20cm

60–100 m
→ L3 / 40cm
```

Configurable.

---

58. Refinement

Lower level number means finer.

Conceptually:

```
desired_level =
distance_level
-
semantic_refinement
-
uncertainty_refinement
-
dynamic_refinement
-
geometry_refinement
```

then clamp.

---

59. Never Split Per Point Immediately

Bad:

```
point
→ split tree

next point
→ split

next point
→ split
```

This creates excessive structural operations.

Instead:

```
points
   ↓
requested resolution aggregation
   ↓
one hierarchy plan
   ↓
batch structural changes
```

---

60. Refinement Request Map

During prepass:

```
unordered_map<CellKey, uint8_t> desired_level;
```

For all requests affecting the same region:

```
keep finest requested level
```

Then apply splits once.

---

61. Information-Density Gate

Refinement only proceeds if observations justify it.

Examples:

```
recent_points >= Nmin
```

or:

```
point_density >= threshold
```

If not:

```
cell becomes uncertain / unknown
```

rather than being subdivided into meaningless empty cells.

---

62. Split Algorithm

```
function split(parent):

    ensure parent is active leaf

    deactivate parent

    allocate 4 children

    replay relevant recent observations

    mark parent has_children

    mark children dirty
```

---

63. Recent Point Replay Architecture

Do NOT duplicate multiple LiDAR frames inside every map tile.

That would waste large amounts of memory.

Instead maintain:

Global Frame Replay Cache

```
Frame N-2

Frame N-1

Frame N
```

plus a lightweight spatial index.

---

64. Replay Cache

Recommended:

```
3 semantic frames
```

stored in reusable ring buffers.

Each cached frame contains:

```
positions

semantics

confidence

uncertainty
```

---

65. Spatial Replay Index

For each replay frame:

```
coarse tile key
→
point indices/ranges
```

When a map cell is split:

```
query overlapping coarse replay buckets
```

instead of scanning every historic point.

---

66. Why Global Replay Is Better

It avoids:

```
same point copied into tile cache

same point copied again elsewhere

large duplicate memory
```

while preserving local re-binning capability.

---

67. Merge Algorithm

Four siblings may merge if:

```
all four active

no active dynamic object

no critical hazard

low uncertainty

similar ground geometry

similar semantic state

stable for N frames
```

Then:

```
aggregate children
     ↓
activate parent
     ↓
deactivate children
```

---

68. Hysteresis

Refine:

```
quickly
```

Coarsen:

```
slowly
```

Example initial values:

```
refine confirmation:
1–2 frames

coarsen stability:
10 frames
```

This prevents:

```
split
merge
split
merge
```

flicker.

---

69. Lazy Maintenance

Do NOT scan every active map cell every frame.

Maintenance SHALL operate on:

```
cells touched this frame

plus

small rotating subset of older tiles
```

---

70. Maintenance Budget

Example:

```
maximum maintenance work:
~1–2 ms / frame
```

Tasks:

```
staleness

coarsening

retirement

confidence decay
```

Large global sweeps are avoided.

---

71. Cell Geometry State

Logical cell contains:

```
ground_mean

ground_variance

z_min

z_max

roughness

overhang_low

overhang_high
```

Research identified a single height per cell as insufficient for bridges, branches and multilevel geometry.

---

72. Cell Semantic State

Store:

```
dominant semantic label

dominant confidence

secondary hazard label

secondary hazard confidence

uncertainty
```

Do not require a full semantic probability vector for every persistent map cell.

---

73. Cell Navigation State

Store or derive:

```
occupancy
```

```
traversability

dynamic probability

unknown state

last observed timestamp
```

---

74. Cell Debug State

For AVR Console's:

    Why This Resolution?

store compact information:

```
last_base_level

final_level

resolution_reason_bits
```

Reason bits:

```
SEMANTIC

UNCERTAINTY

DYNAMIC

GEOMETRY

DENSITY_REJECTED
```

---

75. Resolution Explanation

Example backend result:

```
base level
L3 / 40 cm

final level
```

```
L1 / 10 cm

reasons:
SEMANTIC

class:
POLE

point density:
PASS

uncertainty:
0.09
```

This powers the frontend's unique explainability interface.

---

76. Point Aggregation

Research recommends retaining multiple geometric statistics rather than relying on one mean, because mean, percentiles and variance solve different mapping failures.

Per frame/cell compute:

```
count

z_min

z_max

mean

variance

low percentile

semantic evidence

hazard evidence
```

---

77. Online Variance

Use:

```
Welford's algorithm
```

for numerically stable streaming:

```
mean

variance
```

updates.

---

78. Ground Estimation

Ground candidates originate from:

```
semantic terrain classes

+

geometric continuity
```

Then robust ground observation can use:

```
low percentile

or

median of lowest coherent surface cluster
```

depending on point count.

---

79. Elevation Fusion

Maintain:

```
μ_ground

σ²_ground
```

Use a variance-aware update.

Conceptually:

```
K = prior_variance
    /
    (prior_variance + measurement_variance)

μ_new =
μ_prior
+
K × (z_measurement - μ_prior)
```

---

80. Outlier Gate

Before elevation fusion:

```
|z_measurement - ground_mean|
```

above configured threshold must be treated as:

```
step

obstacle

secondary surface

outlier
```

rather than blindly modifying terrain elevation.

---

81. Semantic Fusion

Use decayed evidence.

Concept:

```
E_c(t) =
λ E_c(t-1)
+
weight × observation_probability
```

Compact implementation may track:

```
dominant class evidence

hazard class evidence
```

rather than every class.

---

82. Small-Object Preservation

Example:

```
95 road returns
3 pole returns
2 curb returns
```

must NOT become:

```
ROAD ONLY
```

Store:

```
dominant = ROAD

secondary_hazard = POLE
```

Then request refinement.

---

83. Geometry Backup

Semantic segmentation can fail.

Therefore a region may also become a hazard/refinement candidate when geometry contains:

```
strong vertical discontinuity

thin vertical cluster

large height range

unexpected step
```

This limits the severity of semantic cascade failure.

84. Vertical Profile

A temporary per-frame cell profile may divide height into small bins.

Example:

```
10 cm vertical bins
```

This is an intermediate calculation only.

It does NOT create a persistent 3D voxel map.

---

85. Overhang Detection

Given:

```
ground cluster

+

upper cluster

+

clear vertical gap
```

set:

```
overhang_low

overhang_high
```

---

86. Ground-Connected Obstacle

If returns form a continuous vertical structure beginning near ground:

```
ground
███
```



classify as:

```
blocked obstacle
```

not overhang.

---

87. Vertical Clearance

```
clearance =
overhang_low
-
ground_mean
```

This becomes a traversability constraint.

---

88. Static vs Dynamic Architecture

The research strongly recommends avoiding permanent fusion of moving objects into static terrain.

AVR backend therefore conceptually maintains:

```
TERRAIN LAYER

STATIC STRUCTURE LAYER

OBJECT OCCUPANCY LAYER

DYNAMIC TRACK LAYER
```

---

89. Why Use an Object Layer?

Consider a parked vehicle.

It is:

```
not moving
```

but it is still:

```
an obstacle
```

Therefore the choice cannot be only:

```
static terrain
vs
moving objects
```

Object occupancy remains separate from terrain.

---

90. Dynamic-Capable Points

Classes:

```
VEHICLE

PERSON

CYCLIST
```

are diverted initially into object processing.

They are not permanently fused into ground terrain.

---

91. Object Clustering

Initial method:

```
DBSCAN
```

Input:

```
dynamic-capable semantic points
```

Output:

```
Detection[]
```

---

92. Range-Aware DBSCAN

Point spacing increases with distance.

Therefore V2 or tuned V1 MAY use:

```
eps(d) =
eps0 + k × distance
```

rather than one fixed clustering radius everywhere.

Start simple before enabling this.

---

93. Detection Structure

```
struct Detection
{
    SemanticClass semantic;

    Eigen::Vector3f centroid;

    Eigen::Vector3f dimensions;

    float confidence;

    uint32_t point_count;
};
```

---

94. Tracker

Use:

```
constant-velocity Kalman Filter

+
```

```
Hungarian assignment
```

AB3DMOT uses this same basic design philosophy—3D Kalman filtering plus Hungarian association—as an efficient baseline.

Implementation should be native in `avr_tracking_core`.

---

95. Association Cost

Initial:

```
distance cost

+

semantic mismatch penalty
```

Optional:

```
3D IoU
```

for better box association.

---

96. Track State

```
x
y
z

vx
vy
vz
```

plus:

```
class

confidence

age
```

```
hits

misses
```

---

97. Moving Decision

Do not classify an object moving from one noisy velocity estimate.

Require:

```
speed > threshold

for multiple observations
```

with hysteresis.

---

98. Object TTL

Object occupancy is short-lived.

Example:

```
0.5–2 seconds
```

configurable.

Terrain lasts much longer.

---

99. Traversability Pipeline

```
adaptive cells
      ↓
metric neighbourhood
      ↓
local terrain fit
      ↓
slope
roughness
step
clearance
```

```
semantic hazard
uncertainty
      ↓
vehicle constraints
      ↓
traversability
```

---

100. Resolution-Independent Neighbour Queries

Traditional fixed grids use:

```
3×3 cells
```

AVR-Map cannot assume that.

Instead query:

```
all active leaves
within radius R metres
```

because neighbours can have different cell sizes.

---

101. Local Plane

Fit:

```
z = ax + by + c
```

using valid neighbouring ground observations.

Prefer weighted least squares.

Weights MAY reflect:

```
ground variance

cell area

observation age
```

---

102. Slope

```
slope =
atan(
sqrt(a² + b²)
)
```

---

103. Roughness

Example:

```
R =
RMS vertical residual
from fitted plane
```

---

104. Step Height

Compare adjacent valid ground surfaces in metric neighbourhood.

Do not use only cell index differences.

---

105. Hard Traversability Veto

Immediately blocked if:

```
occupied obstacle

slope beyond vehicle limit

step beyond capability

insufficient overhead clearance
```

---

106. Unknown Is Not Safe

Insufficient evidence must produce:

```
UNKNOWN
```

not:

```
FREE
```

This applies to:

```
mapping

occupancy

traversability
```

---

107. Map Revision Commit

After all per-frame changes complete:

```
revision++

dirty cells collected

deleted cells collected

metrics finalized
```

Only then publish derived state.

---

108. Map Delta

Never transmit the entire map every frame.

Publish:

```
UPSERT

DELETE
```

changes only.

---

109. Dirty Tracking

Every tile contains:

```
dirty bitset
```

When a cell changes:

```
dirty[index] = true
```

At frame commit:

```
collect dirty cells

emit delta

clear dirty bits
```

---

110. ROS Map Delta Message

Use SoA-style arrays.

Concept:

```
std_msgs/Header header

uint64 revision
uint64 previous_revision

int32[] cell_x
int32[] cell_y
uint8[] level

float32[] ground_z
float32[] z_max
float32[] overhang_low

uint16[] semantic
uint8[] semantic_confidence
uint8[] uncertainty
uint8[] traversability
uint8[] flags
uint8[] resolution_reason
```

```
int32[] deleted_x
int32[] deleted_y
uint8[] deleted_level
```

All upsert arrays must share equal length.

---

111. Snapshot Protocol Inside ROS

Occasionally consumers need a full map.

Do not return hundreds of thousands of cells in one huge ROS service response.

Use:

```
RequestMapSnapshot service
```

which returns:

```
snapshot_id
revision
```

The mapper then publishes:

```
MapSnapshotChunk
```

messages.

---

112. Snapshot Chunks

Each includes:

```
snapshot_id

revision

chunk_index

chunk_count

cells[]
```

This supports:

```
gateway restart

debug export

frontend resynchronization
```

without giant service payloads.

---

113. Critical Snapshot Rule

Snapshot is generated only from a:

```
COMMITTED MAP REVISION
```

Never from an update in progress.

---

114. ROS Topics

Core input:

```
/avr/points_raw

/avr/points_filtered

/avr/points_semantic

/avr/odometry
```

Core output:

```
/avr/map_delta

/avr/map_snapshot_chunk

/avr/tracks

/avr/traversability

/avr/metrics
```

```
/avr/system_status

/diagnostics
```

Visualisation:

```
/avr/viz/semantic

/avr/viz/map

/avr/viz/tracks
```

---

115. Services

```
/avr/reset_map

/avr/query_cell

/avr/request_map_snapshot

/avr/set_mode

/avr/pause

/avr/resume
```

---

116. Query Cell Service

This service powers AVR Console inspection.

Request:

```
CellKey
```

or:

```
x/y position
```

Response:

```
cell key

bounds

resolution

base resolution

resolution reasons

ground

variance

z max

overhang

semantic class

semantic confidence

hazard

uncertainty

traversability

dynamic probability

point count

last observed
```

---

117. Map QoS

Recommended:

Sensor Topics

```
BEST_EFFORT

KEEP_LAST 2

VOLATILE
```

Map Delta

```
RELIABLE

KEEP_LAST 5–10
```

Snapshot Chunks

```
RELIABLE
```

Tracks

```
BEST_EFFORT
KEEP_LAST 2
```

Metrics / Status

```
RELIABLE
```

---

118. ROS Callbacks Must Be Short

Bad:

```
subscription callback
→ run entire map update
→ 30ms
```

Preferred:

```
callback
→ validate
→ move message into bounded mailbox
→ return
```

Worker performs expensive computation.

---

119. Latest-Frame Mailbox

For streams where stale data has little value:

```
PointCloud

Semantic frame
```

use bounded latest-frame storage.

If producer overtakes consumer:

```
old unprocessed frame replaced
```

and:

```
dropped_frames++
```

---

120. Do Not Drop Map Deltas Arbitrarily

Unlike raw point frames:

```
map deltas depend on revision order
```

If consumer misses:

```
revision 842
```

and receives:

```
843
```

it must:

```
resynchronize
```

rather than silently continue.

---

121. Core Concurrency Principle

There shall be:

> **One writer to the hierarchical map.**

This makes:

```
split

merge

fusion

pruning

revision
```

far easier to reason about.

---

122. Map Worker

One dedicated map worker performs:

```
hierarchy mutation

cell fusion

maintenance

revision commit
```

Other threads MAY prepare data.

They do not mutate map topology.

---

123. Tracker Concurrency

Tracking MAY execute concurrently with terrain aggregation because it processes a separate object point subset.

Initial V1 may execute sequentially if profiling shows tracking cost is already low.

Correctness before unnecessary concurrency.

---

124. Executor

`avr_core_container` should use:

```
MultiThreadedExecutor
```

with explicit callback groups.

ROS 2 supports callback-group-level executor control, including separating callbacks with different scheduling requirements.

---

125. Callback Groups

Recommended:

```
ingress group
→ fast subscriptions

control group
→ services / configuration

metrics group
→ timers / diagnostics

snapshot group
→ low-priority snapshot export
```

Map mutation is delegated to the dedicated worker.

---

126. Graceful Shutdown

Workers SHOULD use:

```
std::jthread
std::stop_token
```

where possible.

Shutdown:

```
stop accepting frames

finish/abort current frame safely

flush recorder

close gateway

release GPU

shutdown ROS
```

---

127. Lifecycle Nodes

Critical C++ nodes SHOULD use:

```
rclcpp_lifecycle::LifecycleNode
```

ROS 2 lifecycle provides standard states including Unconfigured, Inactive, Active and Finalized.

---

128. Node Startup

Example:

```
UNCONFIGURED

↓ load configuration

INACTIVE

↓ allocate map/GPU/resources

READY

↓ supervisor activates

ACTIVE
```

Input activation occurs last.

This prevents sensor frames arriving before downstream consumers are ready.

---

129. System Supervisor

Package:

```
avr_supervisor
```

System states:

```
STARTING

READY

RUNNING

PAUSED

DEGRADED

ERROR

STOPPING
```

---

130. Degraded Is Separate From Lifecycle

Example:

```
semantic model fails
```

ROS mapper may remain:

```
ACTIVE
```

while AVR system status becomes:

```
DEGRADED_GEOMETRY_ONLY
```

This is preferable to shutting everything down.

---

131. Failure Isolation Matrix

Semantic Model Failure

```
geometry mapping continues

semantic refinement disabled

UI shows DEGRADED
```

Tracker Failure

```
terrain continues

dynamic-capable object occupancy retained with short TTL
```

Gateway Failure

```
no effect on map core
```

Recorder Failure

```
runtime continues

warning emitted
```

Missing Pose

```
persistent map insertion pauses
```

GPU OOM

```
semantic provider restarts/falls back
```

---

132. Diagnostics

Every major component publishes:

```
OK
```

```
WARN

ERROR

STALE
```

through ROS diagnostics.

ROS provides `diagnostic_updater` utilities for reporting topic/device/system health.

---

133. Diagnostic Fields

Examples:

```
input_hz

last_frame_age

semantic_hz

semantic_latency

mapping_latency

active_cells

tile_count

tracker_tracks

dropped_frames

gateway_clients

replay_cache_frames

map_revision
```

---

134. Metrics vs Diagnostics

Metrics answer:

> How fast is the system?

Diagnostics answer:

Is the system healthy?

Keep them separate.

---

135. Frontend Backend Boundary

AVR Console SHALL NOT connect directly to every ROS topic.

Instead:

```
ROS
 ↓
AVR Gateway
 ↓
clean web protocol
 ↓
AVR Console
```

---

136. Why Use a Gateway?

It gives:

```
stable frontend API

binary point-cloud streaming

map-delta compression

subscription filtering

backpressure

security boundary

ROS isolation

future non-ROS compatibility
```

---

137. Gateway Technology

Recommended:

C++ + Drogon

Reasons:

- C++17/20;
- asynchronous non-blocking networking;
- HTTP;
- JSON REST;
- WebSocket;
- static-file serving;
- Linux support;
- no additional runtime language required.

Drogon currently supports asynchronous HTTP and WebSocket servers in C++17/20.

---

138. Why Not FastAPI for Gateway?

FastAPI is excellent for ordinary APIs.

But this gateway must potentially move:

```
hundreds of thousands of map/point records
```

while sharing C++ data structures and ROS subscriptions.

Python adds no useful advantage here.

Python remains appropriate for:

```
AI inference

experiment tooling
```

---

139. Why Not Node/Express?

Same reason.

There is no benefit in:

```
C++ ROS
→ serialize
→ Node backend
```

```
→ serialize
→ browser
```

when a thin C++ gateway can consume ROS directly.

---

140. Gateway Internal Architecture

```
ROS Subscribers
      ↓
bounded queues / cached latest state
      ↓
Display Mirror
      ↓
Binary Serializer
      ↓
WebSocket Sessions
```

Control side:

```
HTTP request
      ↓
validation
      ↓
ROS service / parameter call
      ↓
response
```

---

141. Read-Only Display Mirror

This is a critical optimization.

`avr_gateway` maintains its own display representation using:

```
/avr/map_delta
```

Therefore:

```
browser connect

browser refresh
```

```
map snapshot request
```

does NOT repeatedly walk and lock the live map engine.

---

142. Gateway Mirror

Stores only frontend-relevant fields:

```
CellKey

ground_z

max_z

overhang_low

semantic

confidence

uncertainty

traversability

flags

resolution reason
```

It does not contain all internal mapping statistics.

---

143. Gateway Cold Start

On startup:

```
request map snapshot

build mirror

subscribe to later deltas
```

Snapshot corresponds to revision `R`.

Then apply:

```
R+1
R+2
...
```

---

144. Revision Gap

If gateway receives:

```
expected 810

received 812
```

it SHALL:

```
discard incremental trust

request new snapshot
```

Never infer missing state.

---

145. Web Backend Routes

Recommended:

```
GET  /healthz

GET  /readyz

GET  /api/v1/system

GET  /api/v1/config

POST /api/v1/config/validate

POST /api/v1/config/apply

POST /api/v1/map/reset

GET  /api/v1/cell/{level}/{morton}

POST /api/v1/map/snapshot
```

```
GET  /api/v1/playback

POST /api/v1/playback/play

POST /api/v1/playback/pause

POST /api/v1/playback/seek

GET  /api/v1/runs

GET  /api/v1/runs/{run_id}

POST /api/v1/benchmark/start

POST /api/v1/benchmark/stop
```

---

146. Health Endpoints

`/healthz`

answers:

```
gateway process alive?
```

`/readyz`

answers:

```
is core system ready to serve meaningful AVR data?
```

These must not mean the same thing.

---

147. WebSocket Endpoint

```
/ws/v1/stream
```

Used for:

```
point cloud

map deltas
```

```
tracks

metrics

system status
```

---

148. Control Plane vs Data Plane

Control Plane

JSON.

Examples:

```
subscribe

unsubscribe

seek

change visualization rate
```

Data Plane

Binary.

Examples:

```
point cloud

map snapshot

map delta
```

---

149. WebSocket Handshake

Client:

```
{
  "op": "hello",
  "protocol": 1
}
```

Server:

```
{
  "op": "hello",
  "protocol": 1,
  "mapRevision": 842,
  "mode": "FULL_ADAPTIVE"
}
```

---

150. Stream Subscription

Client:

```
{
  "op": "subscribe",
  "streams": [
    "points",
    "map",
    "tracks",
    "metrics"
  ],
  "points": {
    "maxPoints": 60000,
    "hz": 10
  },
  "map": {
    "hz": 5
  }
}
```

---

151. Gateway Must Support Per-Client Rate Limits

One browser may want:

```
10Hz point cloud
```

another:

```
2Hz
```

No need to send maximum-rate data to every connected browser.

---

152. Binary Wire Header

Recommended fixed header:

```
struct WireHeader
{
    char magic[4];          // "AVR1"

    uint16_t version;

    uint16_t type;

    uint32_t header_bytes;

    uint32_t payload_bytes;

    uint32_t sequence;

    uint32_t flags;

    uint64_t timestamp_ns;

    uint64_t revision;
};
```

40 bytes.

All multi-byte fields:

```
little-endian
```

for V1.

Protocol version is mandatory.

---

153. Binary Message Types

```
1  POINT_FRAME

2  MAP_SNAPSHOT

3  MAP_DELTA

4  RESERVED
```

Low-volume messages remain JSON initially.

---

154. Point Frame Payload

Prefer structure-of-arrays:

```
uint32 count

float32 x[count]

float32 y[count]

float32 z[count]

uint16 semantic[count]

uint8 confidence[count]

uint8 flags[count]
```

Optional raw intensity only when requested.

---

155. Map Payload

Snapshot/delta cell arrays:

```
int32 cx[]

int32 cy[]

uint8 level[]

float32 ground_z[]
```

```
float32 max_z[]

float32 overhang_low[]

uint16 semantic[]

uint8 semantic_confidence[]

uint8 uncertainty[]

uint8 traversability[]

uint8 flags[]

uint8 reason_bits[]
```

Deletes contain only keys.

---

156. Do Not Send Full Map Each Frame

Initial connection:

```
snapshot
```

After:

```
delta

delta

delta
```

This is essential for frontend efficiency.

---

157. Browser Backpressure

Each client has bounded outbound buffers.

For point frames:

```
newest wins
```

Old point frame may be discarded.

For map deltas:

```
do not silently drop
```

If queue exceeds limit:

```
coalesce safely
```

or:

```
force client snapshot resync
```

---

158. Slow Frontend Rule

A slow browser must NEVER:

```
slow the mapper
```

Gateway subscriptions receive copies/derived state asynchronously.

---

159. Static Frontend Serving

Production demo MAY serve:

```
frontend/dist/
```

directly from `avr_gateway`.

Then the demo needs only one address:

```
http://localhost:8080
```

This also avoids most CORS issues.

---

160. Foxglove

Run:

```
foxglove_bridge
```

as an optional parallel engineering tool.

Foxglove recommends its Bridge over rosbridge for new ROS 2 setups because of performance, features and stability.

---

161. Foxglove Is Not the Custom UI Backend

Use Foxglove for:

```
debugging

topic inspection

quick plots

backup visualization
```

Use AVR Gateway for:

```
AVR Console
```

---

162. Do Not Implement Legacy Foxglove Protocol Yourself

Foxglove's old websocket protocol repository is now archived and explicitly recommends using the current Foxglove SDK instead of implementing the legacy protocol directly.

So:

```
custom AVR protocol
```

for AVR Console,

and:

```
official foxglove_bridge / SDK
```

for Foxglove.

---

163. Storage Philosophy

Use different storage systems for different information.

```
LIVE MAP
→ RAM

RAW / TIME SERIES
→ MCAP

RUN METADATA
→ SQLite

CONFIG
→ YAML

SUMMARIES
→ JSON / SQLite

ANALYSIS EXPORT
→ CSV
```

---

164. No Database in Critical Mapping Path

AVR mapping SHALL NOT perform:

```
SQL insert
```

for each point/cell.

Ever.

The map is an in-memory real-time data structure.

---

165. Sensor Recording

Use:

```
rosbag2
+
MCAP
```

for:

```
raw point cloud

odometry

semantic output

tracks

metrics

diagnostics
```

ROS 2 Jazzy includes the `rosbag2_storage_mcap` storage plugin.

---

166. Experiment Catalog

Use lightweight:

```
SQLite
```

database.

Suggested:

```
results/avr_runs.db
```

---

167. `runs` Table

Concept:

```
run_id

started_at

finished_at

experiment

mode

dataset

sequence

git_commit

config_hash

model_id

hardware_json

status
```

---

168. `run_summary` Table

```
run_id

fps_mean

fps_p05

e2e_p50_ms

e2e_p95_ms

map_p95_ms

mean_active_cells

mean_map_bytes

peak_rss_bytes

small_object_preservation

elevation_rmse
```

```
dropped_frames
```

Nullable when metric doesn't apply.

---

169. Artifact Table

```
run_id

artifact_type

path

sha256
```

Possible artifacts:

```
MCAP

CSV

resolved config

plots

screenshots

summary JSON
```

---

170. Run Directory

Every experiment run gets:

```
results/
└── 2026-09-24_221500_adaptive_semantic/
    │
    ├── manifest.json
    ├── resolved_config.yaml
    ├── hardware.json
    ├── metrics.csv
    ├── summary.json
```

```
    ├── recording.mcap
    └── artifacts/
```

---

171. Reproducibility Metadata

Capture automatically:

```
Git commit

dirty Git state

Ubuntu version

ROS distro

compiler

CPU

RAM

GPU

driver

CUDA

PyTorch

model weights hash

dataset

sequence

config SHA-256
```

---

172. Configuration Architecture

Precedence:

```
compiled defaults

↓
```

```
base YAML

↓

dataset/sensor profile

↓

launch overrides

↓

approved runtime changes
```

---

173. Profiles

Examples:

```
profiles/
├── semantic_kitti.yaml
├── rellis3d.yaml
├── goose.yaml
├── live_os1.yaml
└── demo.yaml
```

---

174. Resolved Configuration

At startup generate:

```
resolved_config.yaml
```

representing what the system actually uses.

Benchmarks reference this file.

---

175. Parameter Types

Two categories:

Runtime Safe

Examples:

```
semantic refinement threshold

uncertainty threshold

TTL

visualisation rate

traversability weights
```

Restart Required

Examples:

```
base cell resolution

tile size

maximum hierarchy depth

canonical map origin behaviour
```

Frontend must know the distinction.

---

176. Configuration Apply Flow

```
user edits

↓

POST /config/validate

↓

backend validates all bounds

↓

show diff

↓
```

```
POST /config/apply

↓

ROS parameter updates

↓

configuration revision++
```

---

177. Atomicity

Never apply:

```
3 out of 5
```

related settings without reporting failure.

For multi-node bundles:

```
validate first

apply

rollback on failure
```

where practical.

---

178. Parameter Validation

Examples:

```
resolution > 0

bands sorted

threshold 0–1

map radius positive

vehicle height positive
```

```
TTL non-negative
```

ROS parameter callbacks should reject invalid values before mutation. ROS 2 exposes parameter event and update mechanisms for monitoring parameter changes.

---

179. Benchmark Architecture

AVR must support:

```
uniform

distance-only

semantic adaptive

semantic + uncertainty

full V1 adaptive
```

using the same dataset.

---

180. Formal Benchmark Rule

DO NOT run:

```
uniform
+
adaptive
```

simultaneously when measuring their absolute performance.

They would compete for:

```
CPU

RAM

GPU

cache
```

and bias results.

181. Formal Benchmark Procedure

Run:

```
variant A
→ record

stop

variant B
→ same sequence
→ record

stop
```

Repeat.

182. Compare Demo Is Different

For judge-facing visualization:

```
uniform + adaptive
```

MAY run simultaneously.

But latency figures from this presentation mode must not become the official benchmark claims.

183. Experiment Runner

Create:

```
tools/avr_experiment_runner/
```

Python is appropriate here.

Input:

```
experiment:
  name: uniform_vs_adaptive
```

```
  input:
    mcap: demo.mcap

  variants:
    - uniform_5cm
    - distance_only
    - semantic
    - full

  repeats: 3

  warmup_frames: 50

  measurement_frames: 1000
```

---

184. Experiment Runner Tasks

Automatically:

```
launch stack

wait READY

play dataset

warm up

measure

stop

collect files

write metadata

calculate summary

repeat
```

---

185. Dataset Playback

Use:

```
use_sim_time = true
```

with controlled rosbag playback.

Data timestamps remain deterministic.

Runtime latency still uses:

```
steady_clock
```

not simulated ROS time.

---

186. Core Benchmark Mode

For serious latency/memory measurement disable:

```
frontend

gateway mirror

RViz

Foxglove

heavy visualization
```

unless specifically measuring complete end-to-end demo overhead.

---

187. GPU Timing

Use:

```
CUDA events
```

for pure network inference timing.

Use:

```
steady_clock
```

for full semantic-node latency including:

```
projection

host/device transfer

inference

back-projection
```

Both numbers are useful.

---

188. Benchmark Statistics

Report:

```
mean

median

p50

p95

p99

standard deviation
```

where sample count permits.

Never report only the best frame.

---

189. Warm-Up

Exclude:

```
initial GPU allocation

model warmup

cache population
```

from steady-state metrics.

---

190. Performance Targets

Initial engineering targets:

```
sensor stream
10 Hz

map update
≤ 20 ms target

tracking
≤ 10 ms target

traversability
≤ 15 ms target

gateway serialization
≤ 5 ms target

end-to-end p95
≤ 100 ms target
```

These are prototype design targets—not benchmark claims.

---

191. Gateway Stream Budget

Example:

```
60k visualization points

xyz float32
+
small semantic metadata
```

at 10 Hz is manageable on localhost.

The gateway SHOULD downsample visualization independently of mapping.

---

192. Important Separation

Backend mapping may process:

```
100k+ points
```

while frontend receives:

```
50k visualization points
```

This does NOT change map accuracy.

It only changes visualization load.

---

193. Map Memory Reporting

Report three things separately:

```
logical active cell bytes

container / allocated map bytes

whole process RSS
```

Do not call all three:

```
"map memory"
```

without distinction.

---

194. Memory Benchmark Isolation

Gateway mirror duplicates some display state.

Therefore:

```
core map memory benchmark
```

must measure map process separately from browser/gateway memory.

---

195. Logging

Use normal ROS logging:

```
DEBUG

INFO

WARN

ERROR

FATAL
```

Avoid per-point logs.

Avoid per-cell logs during ordinary operation.

---

196. Structured Events

Meaningful events should include:

```
event type

timestamp

frame seq

map revision

component

message
```

Examples:

```
SEMANTIC_PROVIDER_LOST

MAP_RESET

SNAPSHOT_STARTED

CONFIG_APPLIED
```

```
FRAME_DROPPED
```

---

197. Logging Hot Path Rule

No string formatting or filesystem writing inside per-point loops.

---

198. Asynchronous Metrics Writer

Benchmark metrics go through:

```
bounded logging queue
```

to a writer thread.

If storage becomes temporarily slow:

```
mapping continues
```

---

199. Profiling Tools

Use:

```
perf

heaptrack

Valgrind where appropriate

AddressSanitizer

UndefinedBehaviorSanitizer

Nsight Systems

ROS 2 tracing
```

ROS Jazzy includes ROS tracing support and the `ros2 trace` tooling.

---

200. Main Optimization Order

Do NOT optimize randomly.

Profile in this order:

```
1 semantic inference

2 PointCloud2 parsing

3 point-to-cell lookup

4 hierarchy split/merge

5 aggregation

6 traversability neighbourhoods

7 serialization

8 visualization publishing
```

---

201. CPU Optimisation Rules

Prefer:

```
batch loops

contiguous vectors

reserve()

reused buffers

integer cell arithmetic

dirty masks

tile iteration
```

Avoid:

```
per-point heap allocation
```

```
std::map in hot paths

string operations

virtual calls per point

recursive pointer trees
```

---

202. Floating Point Policy

Use floating point for:

```
physical positions

geometry

probabilities
```

Use integer arithmetic for:

```
cell ownership

hierarchy

indices

boundaries
```

This is one of the key correctness rules.

---

203. Map Mutation Policy

Only map worker mutates:

```
active masks

cell values

hierarchy

tile store
```

This avoids large amounts of locking.

---

204. Read Access

Other backend consumers should receive:

```
deltas

metrics

queries
```

rather than directly sharing mutable map memory.

---

205. Frontend Queries

Cell query is asynchronous.

If it takes:

```
20–50ms
```

that is acceptable.

It is not in the LiDAR critical path.

---

206. Snapshot Priority

Snapshot generation is:

```
low priority
```

and can be delayed by active frame processing.

Real-time mapping always wins.

---

207. Security Model

Default bind:

```
127.0.0.1
```

for gateway.

No external network exposure required.

---

208. LAN Demo Mode

If binding:

```
0.0.0.0
```

enable:

```
shared bearer token

allowed-origin restriction
```

at minimum.

---

209. Do Not Expose Arbitrary ROS Control

Browser SHALL NOT be able to:

```
publish arbitrary ROS topics

call arbitrary ROS services

execute commands

read arbitrary filesystem paths
```

Gateway exposes only explicitly permitted operations.

---

210. API Input Limits

Validate:

```
WebSocket message size

JSON payload size

seek range

configuration ranges

snapshot frequency
```

Prevent one browser from exhausting system memory.

---

211. Static Files

Gateway must prevent:

```
../
```

path traversal when serving frontend/artifacts.

---

212. Testing Pyramid

```
          E2E
         /
    INTEGRATION
       /
    UNIT + PROPERTY
```

Most tests belong at the bottom.

---

213. Unit Tests

Mandatory areas:

```
floor_div negatives

Morton encode/decode

parent-child math
```

```
half-open boundaries

tile indexing

split

merge

single-leaf ownership

Welford variance

semantic fusion

hazard retention

overhang detection

Kalman prediction

association

traversability veto
```

---

214. Property / Invariant Tests

Generate random coordinates.

Verify:

```
exactly one leaf owns point

parent boundaries contain children

split followed by valid merge preserves footprint

Morton decode(encode(x,y)) = x,y
```

---

215. Map Revision Tests

Ensure:

```
revision increments once per commit
```

```
no revision on failed frame

delta.previous_revision matches

snapshot revision is consistent
```

---

216. Gateway Tests

Test:

```
handshake

protocol mismatch

snapshot

delta

resync

slow client

disconnect

malformed message

oversized message

cell query
```

---

217. Fault Injection

Test deliberately:

```
missing TF

semantic delay

semantic crash

GPU unavailable

tracker exception
```

```
gateway crash

slow filesystem

dropped map delta

invalid configuration

corrupted WebSocket packet
```

---

218. ROS Integration Testing

Use:

```
launch_testing

launch_pytest
```

for multi-process tests.

ROS Jazzy's launch testing framework can launch processes, inspect exit status and interact with running ROS nodes.

---

219. Sanitizers

CI/debug builds SHOULD include:

```
AddressSanitizer

UndefinedBehaviorSanitizer
```

ThreadSanitizer should be run selectively on C++ components that support it.

---

220. CI Pipeline

On every pull request:

```
format check

C++ compile
```

```
Python lint

unit tests

integration tests

sanitizer subset

frontend contract tests
```

---

221. C++ Quality Tools

Recommended:

```
clang-format

clang-tidy

cpplint optional

GoogleTest
```

Compile warnings:

```
-Wall
-Wextra
-Wpedantic
```

---

222. Python Quality

Recommended:

```
Ruff

mypy where practical

pytest
```

---

223. Build Profiles

Development:

```
RelWithDebInfo
```

Formal benchmarks:

```
Release
```

Debug correctness:

```
Debug + sanitizers
```

---

224. Launch Modes

Create dedicated launch files:

```
dataset_gt.launch.py

dataset_ai.launch.py

live_ai.launch.py

geometry_only.launch.py

compare_demo.launch.py

benchmark.launch.py
```

---

225. Dataset-GT Mode

```
dataset

→ ground truth semantics

→ AVR map
```

Used primarily for algorithm correctness.

226. Dataset-AI Mode

```
dataset LiDAR

→ neural semantics

→ AVR map
```

Used for realistic end-to-end performance.

227. Geometry-Only Mode

```
LiDAR

→ geometry

→ terrain/map
```

Used as:

```
fallback

debug mode
```

228. Compare Demo Mode

Runs:

```
uniform baseline

+

adaptive map
```

for synchronized frontend comparison.

Not official latency benchmarking mode.

229. Backend Bringup

One command:

```
ros2 launch avr_bringup demo.launch.py
```

should start all required robotics processes.

One separate command MAY start frontend development server.

Production gateway may serve built frontend automatically.

---

230. Startup Sequence

```
1 supervisor

2 gateway

3 input configured

4 preprocessing configured

5 semantic model loaded/warmed

6 core map configured

7 tracker/traversability ready

8 recorder optional

9 activate consumers

10 activate input

11 system → RUNNING
```

---

231. Shutdown Sequence

Reverse:

```
input stops
```

```
finish current map transaction

publish final metrics

stop recorder

flush experiment DB

close clients

release GPU/map

shutdown ROS
```

---

232. Docker Strategy

Docker is useful for reproducibility but must not complicate the hackathon demo.

Recommended:

```
native Ubuntu ROS core

+
isolated Python venv/container for AI if necessary
```

Full containerization can be added after pipeline stability.

---

233. Optional Containers

```
avr-core

avr-semantic

avr-gateway
```

If Docker is used for ROS processes:

```
host networking
```

will normally simplify DDS discovery in the prototype environment.

---

234. GPU Container

If semantic inference is containerized:

```
NVIDIA Container Toolkit
```

must be configured.

Test performance against native deployment before final demo.

---

235. Backend-to-Frontend Contract Versioning

Three versions remain explicit:

```
ROS message version

AVR web protocol version

configuration schema version
```

Do not silently change interfaces.

---

236. Web Protocol Compatibility

If:

```
frontend expects protocol 1

gateway supports only protocol 2
```

connection must fail clearly:

```
PROTOCOL_VERSION_MISMATCH
```

not partially render corrupt state.

---

237. Configuration Schema

Add:

```
schema_version
```

to YAML.

Example:

```
schema_version: 1
```

Migrations can then be handled explicitly later.

---

238. System Metrics Structure

`MapMetrics` should contain:

```
frame_seq

map_revision

FPS

input latency

preprocess latency

semantic latency

map latency

tracking latency

traversability latency

end-to-end latency

active tiles

active cells

cells by level

map bytes

process RSS

tracks
```

```
dropped frames
```

---

239. Map Efficiency Metric

```
cell_reduction =
1 -
adaptive_cells / uniform_cells
```

Report as percentage.

---

240. Memory Reduction Metric

```
memory_reduction =
1 -
adaptive_memory / uniform_memory
```

Only compare equivalent extents and map layers.

---

241. Small-Object Metric

```
preservation =
objects represented after mapping
/
objects present before map aggregation
```

Break down by:

```
class

distance

resolution
```

---

242. Performance SLO Philosophy

Prototype SLOs are not hard safety guarantees.

They are:

```
engineering targets
```

The UI must distinguish:

```
TARGET

MEASURED
```

---

243. Backend Event Model

Important events:

```
FRAME_ACCEPTED

FRAME_DROPPED

MAP_COMMITTED

CELL_REFINED

CELL_COARSENED

TRACK_CREATED

TRACK_LOST

SYSTEM_DEGRADED

SYSTEM_RECOVERED

CONFIG_CHANGED
```

Avoid emitting refinement events for every cell into normal logs.

Aggregate them.

---

244. Frame Metrics

Per frame:

```
refined_cells

coarsened_cells

new_tiles

retired_tiles

point_count

ground_points

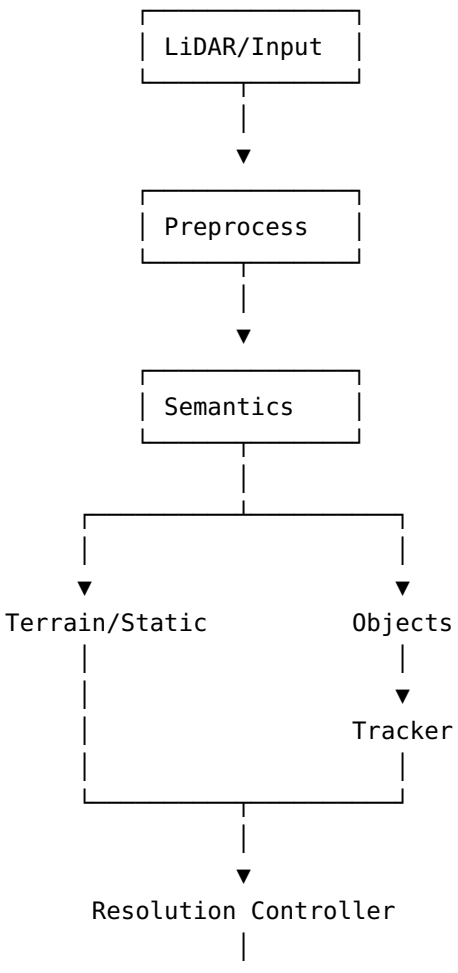
object_points

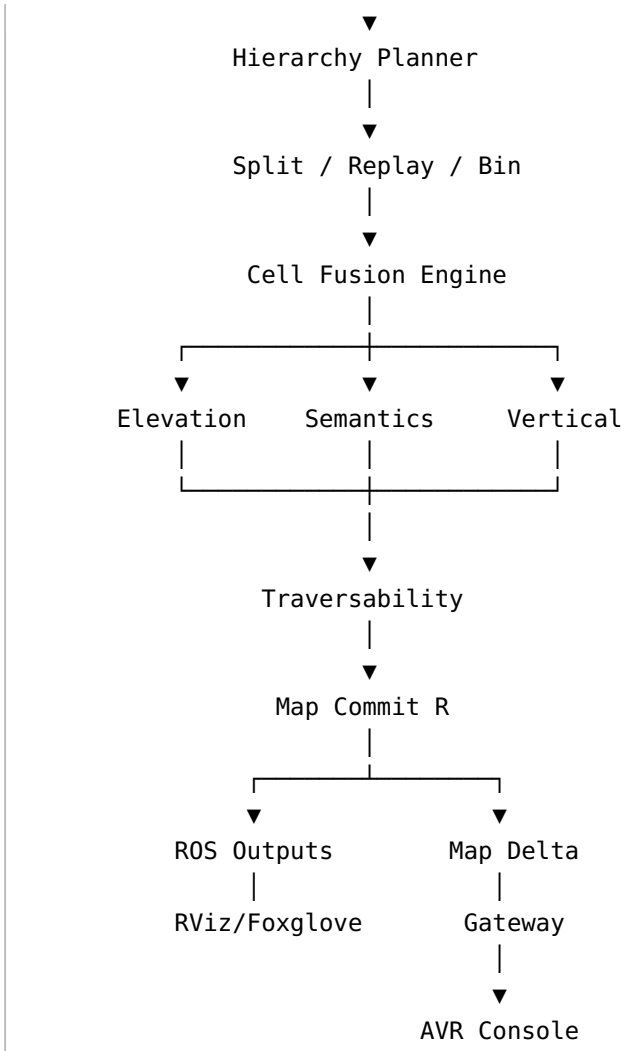
unknown_points
```

Useful during debugging and ablations.

---

245. Data Flow Summary





---

246. Performance-Critical Path

Critical:

```
LiDAR

→ preprocess

→ semantic output

→ map update

→ safety/traversability state
```

Non-critical:

```
web serialization

charts

database

screenshots

run browsing

Foxglove

CSV writing
```

Never confuse them.

---

247. Backend Technical Stack

Robotics

```
ROS 2 Jazzy
rclcpp
rclpy
tf2
rosbag2
MCAP
diagnostic_updater
```

Core

```
C++17/20
Eigen
PCL where useful
STL
```

AI

```
Python
PyTorch
CUDA
SalsaNext V1
```

Gateway

```
C++
Drogon
WebSocket
REST
```

Storage

```
RAM
MCAP
SQLite
YAML
CSV/JSON
```

Testing

```
GoogleTest
pytest
launch_testing
Playwright contract tests
```

Profiling

```
perf
heaptrack
Nsight
ros2_tracing
```

---

248. Technologies Explicitly Not Needed

Do NOT add merely because they are popular:

```
MongoDB

PostgreSQL

Redis

Kafka

RabbitMQ
```

```
Kubernetes

GraphQL

microservice mesh
```

None solves the main AVR prototype problem.

---

249. Backend Architecture Style

This project should be described as:

> **A modular real-time robotics system with process isolation around expensive or failure-prone boundaries—not a conventional web microservices backend.**

Inside the core:

```
modular monolith / composed components
```

is preferable to unnecessarily distributed services.

---

250. Biggest Backend Mistakes to Avoid

Do NOT:

```
put points in a database

send full maps every frame

store map cells in JSON

put map state in Python dictionaries

build a pointer-heavy quadtree

allow browser clients to backpressure ROS

use latest-frame semantics as permanent truth

fuse moving objects into terrain

treat unknown as free
```

```
measure latency with wall clock

benchmark with debug builds

claim memory savings from theoretical math alone

put all code inside ROS callbacks

hard-code dataset class IDs

couple map engine to SalsaNext

couple map engine to frontend
```

---

251. Implementation Order

Backend development should occur in this order:

```
1  common types

2  spatial library
   floor_div
   Morton
   hierarchy math

3  uniform map baseline

4  adaptive tile store

5  distance-only hierarchy

6  frame aggregation

7  elevation fusion

8  map delta / revision system

9  dataset GT semantics

10 semantic refinement

11 replay cache

12 small-object preservation

13 uncertainty refinement
```

```
14 traversability

15 vertical/overhang processing

16 object clustering

17 tracker

18 dynamic object layer

19 metrics / diagnostics

20 gateway

21 frontend protocol

22 semantic neural provider

23 experiment runner

24 MCAP / SQLite run storage

25 optimization / profiling
```

---

252. Why Gateway Comes Late

Frontend mock data already allows UI development.

The mapping core should not be delayed because:

```
HTTP endpoint
```

is unfinished.

---

253. Definition of Backend Done

The backend is complete when one launch command can process a known dataset and produce:

```
semantic point stream

adaptive map

map deltas
```

```
resolution reasons

elevation

small hazard retention

overhang information

tracked objects

traversability

metrics

diagnostics
```

while:

```
frontend connects/disconnects safely

map remains bounded

semantic process can fail without crashing core

slow browser does not slow map

run results are reproducible
```

---

254. Final Acceptance Conditions

Correctness

```
one active leaf per location

no parent/child double ownership

negative coordinates correct

map revisions consistent
```

Adaptivity

```
different hierarchy levels present
```

```
critical semantics can refine cells

uncertainty can trigger refinement

insufficient density prevents fake detail
```

Safety Information

```
small hazards retained

unknown distinct from free

overhang clearance represented

dynamic objects isolated from terrain
```

Runtime

```
sustained target input rate

bounded queues

bounded map memory

no unbounded history
```

Resilience

```
gateway crash ≠ core crash

UI crash ≠ core crash

semantic failure → degraded operation
```

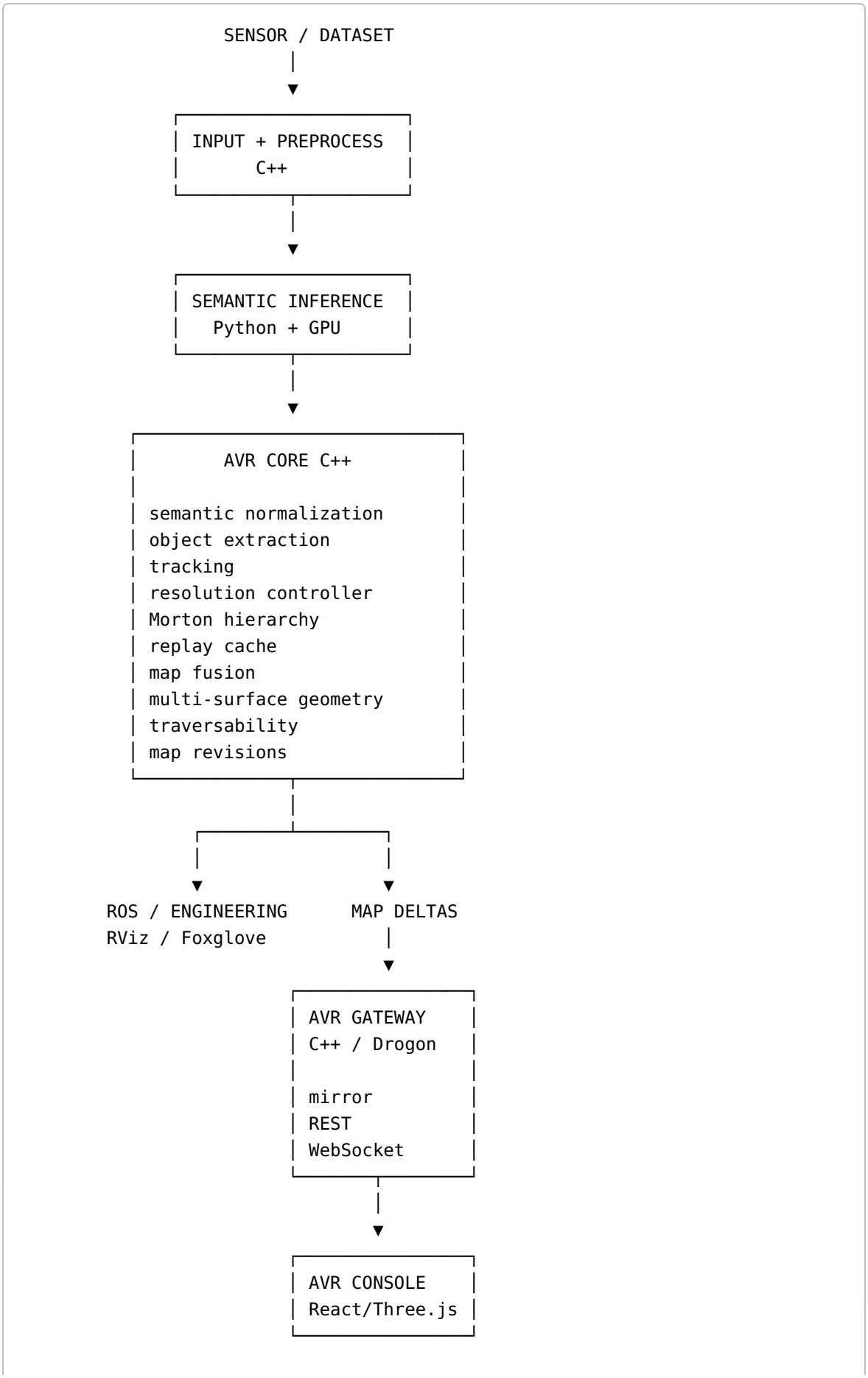
Evidence

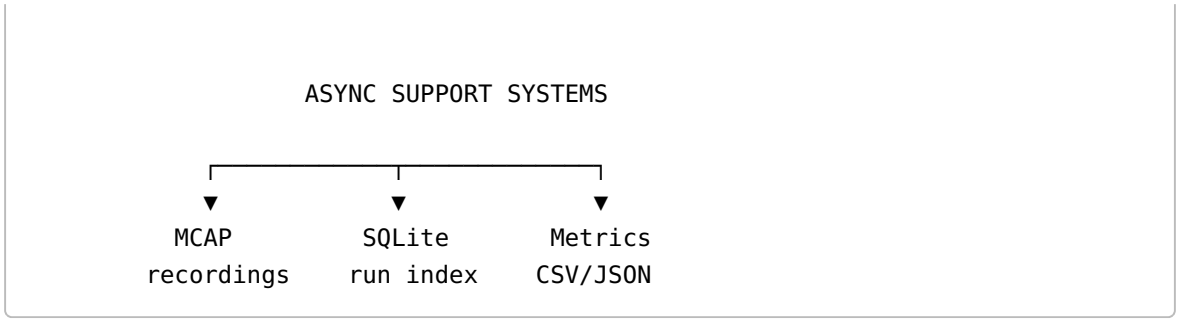
```
uniform vs adaptive measurements reproducible
```

---

255. Final Backend Architecture

The production-shaped SIH prototype backend is therefore:


SENSOR / DATASET
INPUT + PREPROCESS
C++
SEMANTIC INFERENCE
Python + GPU
AVR CORE C++
semantic normalization
object extraction
tracking
resolution controller
Morton hierarchy
replay cache
map fusion
multi-surface geometry
traversability
map revisions
ROS / ENGINEERING
RViz / Foxglove
MAP DELTAS
AVR GATEWAY
C++ / Drogon
mirror
REST
WebSocket
AVR CONSOLE
React/Three.js



---

256. Final Design Principle

The most important backend rule for every teammate is:

> **No feature is allowed to make the mapping engine wait unless that feature is required to produce the map correctly.**

The LiDAR perception path must remain protected from:

```
UI

network

disk

database

visualization

analytics

slow clients
```

That separation is what allows AVR-Map to behave like a robotics system rather than a normal web application.

---

257. Final Backend Definition

AVR-Map's backend is:

> **A bounded-memory, revisioned, failure-isolated, real-time ROS 2 perception pipeline built around a single-writer sparse hierarchical 2.5D mapping engine, with replaceable semantic inference, multi-surface terrain representation, object-layer tracking, vehicle-aware traversability, reproducible experimentation, and a separate high-performance binary gateway for the AVR Console.**

This architecture gives the prototype a credible path from:

```
SIH DEMO
```

to:

```
RESEARCH PROTOTYPE
```

without requiring the backend to be redesigned from scratch.